CS340 - Analysis of Algorithms

Final Exam Review

Announcements:

Office Hour Change:

- Friday cancelled
- Thursday 2-5 instead (Shared with data structures. Priority early)

No lab Tomorrow

Final Exam Options:

1. Dec 12 1-4pm Park 230
2. Dec 15 9:30-12:30 Park 159

Subset Sum

Given a set of positive integers (stored in an array A[1..n]) and an integer X, decide whether any subset of A sums up to X.

Show that this greedy algorithm is incorrect by giving a counterexample.

Greedy Algorithm A:

```
Sort A in decreasing order
remaining  = X, s = {}
for each num in sorted A:
        If num <= remaining:
                s.add(num)
                remaining -= num
                If remaining == 0: return True
return False
```

Subset Sum

Divide and Conquer Algorithm B: Split the array into a left half and a right half and recursively computes SSL = SS(A[0..mid], X) and SSR = SS(A[mid..n], X).

The base case (when the size of the array is 0), returns true iff X == 0.

If SSL = True or SSR = True, return True. Otherwise, return false.

```
SS(A[1..n], X):
    if n == 0:
        return (X == 0)

    let mid = n/2
    let Left  = A[1..mid]
    let Right = A[mid+1..n]

    if SS(Left, X) == true:
        return true
    if SS(Right, X) == true:
        return true
    return false
```

Subset Sum

Divide and Conquer Algorithm C. Split the array into a left half and a right half and recursively computes SSL(A[0..mid], X/2) and SSR(A[mid..n], X/2).

The base case (when the size of the array is 0), returns true iff X == 0.

If SSL = True and SSR = True, return True. Otherwise, return false.

Is this algorithm correct? Either prove correctness or give a counterexample and justify why the algorithm fails.

```
SS(A[1..n], X):
    if n == 0:
        return (X == 0)

    let mid = n/2
    let Left  = A[1..mid]
    let Right = A[mid+1..n]

    if SS(Left, X/2) == true:
        return true
    if SS(Right, X/2) == true:
        return true
    return false
```

Subset Sum - DP

Dynamic Programming Algorithm D: Design a DP algorithm for Subset Sum. Give a clear description of the algorithm and/or pseudocode. **State** and explain the recurrence. **Analyze** the running time of your algorithm. **Prove** that your DP algorithm is correct.

```
OPT(i, j) = OPT(i-1, j) OR OPT(i-1, j-A[i])
```

```
if i == 0 and j == 0 -> TRUE
```

```
if i == 0 and j != 0 -> FALSE
```

Subset Sum - DP

```
def SS(i, j):

    if i == 0 and j == 0 -> TRUE

    if i == 0 and j != 0 -> FALSE


    return SS(i-1, j) OR (x-A[i] >= 0 AND SS(i-1, x-A[i]))
```

Subset Sum - DP

```
Define nxX table M

def SS(i, j):
    if M[i,j]: return M[i.j]
    if i == 0 and j == 0 -> TRUE
    if i == 0 and j != 0 -> FALSE

    M[i,j] = SS(i-1, j) OR (x-A[i] >= 0 AND SS(i-1, x-A[i]))
    return M[i,j]
```

Subset Sum - DP

Runtime?
O(nX)

```
Define nxX table M

def SS(A, X):
    for i in range(0, n+1):
        for j in range(0, X):
            if i == 0 and j == 0: M[i,j] = T
            else if i == 0 and j != 0: M[i,j] = F
            Else:
                M[i,j] = M[i-1, j] OR (x-A[i] >= 0 AND M[i-1, x-A[i]])
    return M[i,j]
```

Subset Sum - DP

Proof of correctness (by induction)

Base Cases: i=0, j= 0 -> True (empty set)

i=0, j != 0, -> False because A contains positive numbers

IH: Assume SS correctly returns whether a subset of A[1...k] for k > 0 and k < i and j >= 0 and j <= X

We will use IH to show that SS returns correct boolean value for k equal to i.

If the ith element in A is not included in the subset, then the subset A[1...i-1] sums to j. By the IH, this is computed correctly

If the ith element in A *is* included in the subset, then the subset A[1..i-1] sums to j - A[i]. Since A consists of positive numbers, j - A[i] must be > 0. If it is positive, by the IH, A[1...j-A[i] is computed correctly.

Sports Elimination Problem

- You are the commissioner of a small sports league. The league has a set of teams S = {1, 2, ..., n}. For each team x in S, its current number of wins is wx. For every distinct pair of teams (x,y) in S, there are g_{xy} >= 0 remaining games to be played between teams x and y this season (assume $g_{xy} = g_{yx}$ and no team plays itself).

- You are particularly interested in a specific team z in S. You want to know whether it is still possible for team z to finish the season in (at least) first place, under some assignment of outcomes to the remaining games. Assume there are no ties in individual games (every game produces exactly one winner), and “first place” means that no other team ends with strictly more wins than team z (ties for first are allowed unless otherwise specified).

Sports Elimination Problem

- Assume team z wins all remaining games
- Team z wins m games where $m = w_z + \sum g_{zy}$

- Model if it is possible to carefully allocate the wins to remaining games z is not playing in, such that no other team ends up with wins > m
  - Use capacity constraints to ensure this

Sports Elimination Problem

Description of the Reduction

- V =
  - Source s and sink t
  - A “game node” (x,y) for each pair (x,y) x != z, y != z and $g_{xy} > 0$
  - A “team node” for each team x != z
- E =
  - Add edges between s and (x,y) with c(s, (x,y)) = g_{xy}
  - Add edges between (x,y) and x and y with c((x,y), x) and c((x,y), y) = g_{xy}
  - Add edges between x and t with c(x,t) = m - w_x
- Team z can be in first place iff |f| = total number of games remaining where z is not playing

Sports Elimination Problem

Runtime Analysis

- |V| =
  - Game nodes: $O(n^2)$
  - Team nodes: n -1 = O(n)
  - Source and sink: 2
  - = $O(n^2)$
- |E| =
  - Between s and game nodes: $O(n^2)$
  - Between game and team nodes: $2*O(n^2)$
  - Between team nodes and t: O(n)
  - = $O(n^2)$
- Runtime of creating the graph = $O(n^2)$
- C = total number of remaining games where z is not playing
  - $\sum g_{xy}$ where x and y are not equal to z
- $O(Cm) = O(\sum g_{xy} n^2)$

Sports Elimination Problem

Proof of correctness

Direction 1: Given an assignment of remaining games, such that no team ends up with more than m wins,

- For each game between x and y, route a unit of flow through the winner's team node
- Each team receives flow equal to their wins among non-z teams
- This flow satisfies capacity constraints since each team cannot win more than g_{xy} games and because this is a valid assignment, they cannot win more than $m - w_x$ games
- It also satisfied conservation since there is one winner to each game

Sports Elimination Problem

Proof of correctness

Direction 2: Given a max flow $f \geq \sum g_{xy}$ where x and y are not equal to z

- The flow into/out of team node x = # of games x wins (among non-z games)
- Since $c(x,t) = m - w_x$, team x wins at most $m - w_x$ in their remaining games
  - We assume they lose all remaining games with z
- Each team x has at most $w_x + (m - w_x) = m$ wins
- Team z has exactly m wins (by definition of m)
- Therefore, z at least ties for first place.

Sports Elimination Problem

- After thinking about it more, you realize that tying for first place might not be good enough: you now want to require that team z finishes strictly in first place. at the end of the season, z must have more wins than every other team (no ties allowed). Modify your construction and/or algorithm from part (1) so that it decides whether team z can still finish strictly in first place. You only need to give a clear description of the changes (no full proof of correctness is required). If the runtime analysis changes, give a brief description of how.

- For each edge (x,t) $C(x,t) = M - (w_x+1)$

ZC is NPC

The zero cycle problem (ZC) decides whether there exists a simple cycle consisting of at least three edges whose total weight is zero.

What does it mean for a problem p to be NPC?

1. $p \in NP$
2. $\forall\ j \in NP$, j reduces to p (in polynomial time)

ZC is NPC

1. $p \in NP$

```
def isValid(E'):
    if |E'| < 3: return False
    cost = 0
    for edge e in E':
        cost += w(e)

    return cost == 0
```

O(m)

ZC is NPC

Reduction Design:

From Hamiltonian Cycle

$V' = \{V \setminus u\} \cup \{u_1, u_2\}$

Keep all edges the same.

Any edge between u in G should be connected to both u_1 and u_2 in G'

$w(u_1, u_2) = -n$ where $n = |V|$

Weight of all other edges in 1

ZC is NPC

Direction 1: We will show that if G has an HC, G' has a ZC

Let C be the set of edges that make the HC in G

By definition of HC, it must pass through n vertices with n edges

In G', replace the edges ...a,u,b... in the cycle with ....a,u_1, u_2,b...

this set of edges has cost n

Adding the edge (u_1, u_2) makes cost = 0 and this is a valid cycle

ZC is NPC

Direction 2: We will show that if G' has a ZC, G has an HC

Let C be a zero cycle in G'

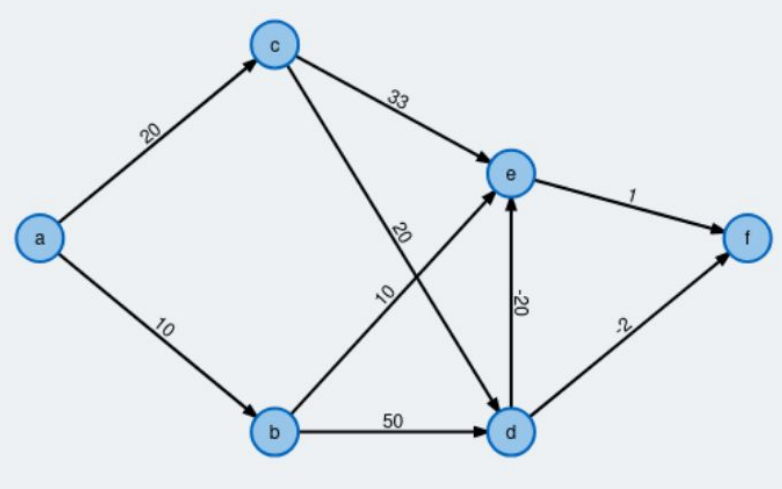
Replace ...a,u_1, u_2,b.. with ...a,u,b...

Because all edges are positive except (u_1, u_2), we know it must go through (u_1, u_2) and exactly n other edges.

If we go through n edges, we touch exactly n vertices and this is a HC in G.

Bellman-Ford Trace

Run the Bellman-Ford algorithm on the following graph. During each iteration, show how the dp matrix changes. List the resulting distances from a to all other nodes




```
BellmanFordDP(G=(V,E), s):
      // dp[i][v] = shortest distance to v using at most i edges
      n = |V|
      m = |E|

      // Initialize DP table
      for i = 0 to n-1:
        for each vertex v:
            dp[i][v] = ∞

      dp[0][s] = 0

      // Fill DP table: n-1 iterations
      for i = 1 to n-1:
        // Copy previous values first
        for each vertex v:
            dp[i][v] = dp[i-1][v]

        // Relax all m edges
        for each edge (u, v) in E:
          if dp[i-1][u] != ∞:
            dp[i][v] = min(dp[i][v], dp[i-1][u] + w(u,v))

      return dp[n-1]
```